What are researchers doing right now to make large language model inference faster and cheaper?

GraphIV — deep research

August 27, 2026

Abstract

(round-1 saturation overridden — one more look) The review is exceptionally thorough, providing a complete chronological evolution from foundational scaling and training techniques to the most recent 2026 frontier methods like cascading architectures and reasoning-aware optimization.

1 Scaling Efficiency: A Survey of Optimization Strategies for Large Language Model Inference

The rapid proliferation of large-scale Transformer architectures has fundamentally altered the landscape of artificial intelligence, yet the massive computational requirements of these models present a significant barrier to widespread deployment. As model parameters continue to grow, the latency and memory demands of inference become increasingly prohibitive, creating a critical need for methods that reduce both time and cost. Addressing these inefficiencies is essential for transitioning from research-scale experimentation to sustainable, real-world applications that require high throughput and low operational overhead.

This review traces the trajectory of optimization research, beginning with the foundational architectures and the distributed training paradigms that define modern large-scale models. We first examine model-centric approaches to efficiency, specifically focusing on compression techniques such as quantization, pruning, and distillation. The discussion then transitions to structural and algorithmic improvements, covering memory-efficient attention mechanisms and the serving systems that manage them, as well as methods to increase throughput through speculative and parallel decoding.

Finally, we explore the evolution from individual model optimizations to complex system-level strategies, including advanced scheduling and disaggregated inference architectures. The review concludes by examining the current frontier of the field, where research is shifting toward reasoning-aware and cost-efficient inference methods designed to optimize the trade-off between computational expenditure and the depth of model reasoning.

1.1 Foundations of Large-Scale Transformer Architectures

The evolution of large language models (LLMs) is rooted in the transition from recurrent and convolutional architectures to multi-head self-attention mechanisms [[1]]. This foundational shift

was primarily driven by the need to improve training time and enable greater parallelization [[1]], providing the structural basis for bidirectional understanding [[2]] and the unification of various NLP tasks into a single text-to-text framework [[3]].

However, a fundamental tension has emerged between architectural efficiency and the pursuit of model capability through massive scaling. While the original Transformer [[1]] aimed to optimize training through parallelization, subsequent research has frequently prioritized parameter count to achieve task-agnosticism. For example, scaling to 175B parameters enables "in-context learning," allowing models to adapt to tasks via text prompts without the need for gradient updates or fine-tuning [[4]]. This scaling approach has yielded significant performance benchmarks, including 86.4%† accuracy on LAMBADA and a 20.50 perplexity on PTB [[4]].

Despite these gains, this scaling trend is inherently counter-productive to the goals of making inference faster and cheaper. The massive parameter counts and large context windows required for such performance significantly increase computational demands [[4]], a challenge mirrored in other large-scale efforts such as the 176B-parameter multilingual BLOOM model [[5]]. Consequently, the research focus has had to bifurcate: while one path seeks to refine the core attention mechanism, another must focus on mitigating the costs of scale through advanced model parallelism.

To address the resource intensity of these massive architectures, researchers have developed optimization methods at the deployment layer. Instead of relying solely on architectural changes, techniques such as tensor parallelism, pipeline parallelism, and interleaved schedules are used to optimize GPU memory and compute operations within GPU clusters [[6]]. Thus, while the foundational Transformer [[1]] provided the mechanism for parallelization, the sheer scale of modern models [[4], [5]] has shifted the optimization burden toward distributed computing strategies to manage the resulting memory and compute overhead [[6]].

Architectural innovations continue to focus on reducing memory bandwidth bottlenecks, such as the transition from multi-head to grouped-query attention (GQA) to optimize key-value loading ([7]). Additionally, parameter-efficient fine-tuning via low-rank decomposition (LoRA) allows for the deployment of specialized models without the overhead of full-parameter updates ([8]).

1.2 Parallelism and Distributed Training Paradigms

The evolution of large language model (LLM) scaling has transitioned from manual architectural partitioning to automated, multi-dimensional parallelism strategies. Foundational methods such as GPipe ([9]) addressed single-accelerator memory limits by utilizing pipeline parallelism to partition a network into distinct computational units across multiple devices ([9]). While GPipe focuses on inter-layer partitioning, Megatron-LM ([10]) introduced intra-layer model parallelism, which splits weight matrices and inputs within self-attention and MLP blocks to manage memory constraints in large-scale multi-node GPU clusters ([10]).

As research progressed, the focus shifted from mere memory capacity to optimizing per-iteration latency and throughput. TeraPipe ([11]) advanced the pipeline paradigm by introducing token-level pipeline parallelism. By employing a dynamic programming algorithm to determine optimal slicing, TeraPipe achieved a 5.0x throughput increase compared to its baseline ([11]). This granularity allows for more efficient resource utilization than the layer-level partitioning seen in GPipe.

To manage the complexity of combining these various forms of parallelism, Alpa ([12]) introduced an automated approach. Rather than requiring manual configuration, Alpa uses compiler passes on Jaxpr and HLO to generate execution plans that unify data, operator, and pipeline parallelism

([12]). This hierarchical strategy, which extends the token-level concepts of TeraPipe, provides a more scalable framework for inter- and intra-operator parallelism ([12]). Finally, the research has moved toward the specific requirements of inference serving. AlpaServe ([13]) builds upon the automated foundations of Alpa ([12]) by applying statistical multiplexing and novel model placement algorithms to minimize stage imbalance, specifically targeting the attainment of Service Level Objectives (SLOs) and reduced serving latency ([13]).

| Method | Optimization Layer | Efficiency Goal | Technical Approach | | :— | :— | :— | :— | | [9] | Architecture | Memory | Pipeline parallelism | | [10] | Architecture | Memory | Intra-layer model parallelism | | [11] | Architecture | Latency (5.0x vs baseline) | Token-level pipeline parallelism | | [12] | Architecture | Speedup/Scaling | Automated hierarchical parallelism | | [13] | Engine | Serving latency/SLO | Statistical multiplexing |

The progression of these methods demonstrates a clear trajectory from manual, memory-centric architectural partitioning ([9], [10]) toward automated, latency-centric systems ([12], [13]). While early methods provided the necessary scale for distributed training, they lacked the granularity and automation required for modern, dynamic workloads. TeraPipe ([11]) addressed this via token-level granularity, which Alpa ([12]) then integrated into a broader, compiler-driven framework. This evolution culminates in AlpaServe ([13]), which shifts the optimization target from training throughput to the real-world constraints of inference serving, such as SLO attainment.

1.3 Model Compression: Quantization, Pruning, and Distillation

Current research into making large language model (LLM) inference faster and cheaper focuses on three primary vectors: architectural distillation, weight quantization, and layer-skipping strategies. These methods target different layers of the model to reduce computational expense, memory footprint, and latency.

Knowledge distillation remains a foundational approach for reducing model complexity by training smaller student models to replicate the soft targets of larger teacher ensembles [[14]]. This architectural-level optimization aims to decrease the overall computational cost and improve deployment feasibility [[14]]. A prominent application of this is DistilBERT, which utilizes a triple loss function to create a version of BERT that is 60% faster at inference time, specifically targeting deployment under constrained computational budgets or on-the-edge devices [[15]]. While distillation effectively reduces the model size and complexity, it requires a significant retraining or fine-tuning phase to transfer knowledge from the teacher to the student.

In contrast to distillation, which modifies the architecture, quantization methods target the weights to reduce storage and computational overhead without necessarily changing the model's structure. GPTQ introduces an accurate post-training quantization technique for generative pre-trained transformers, focusing on reducing the costs associated with storage and multiplications [[16]]. Building upon the need for even more efficient serving, AWQ (Activation-aware Weight Quantization) optimizes for memory size, memory bandwidth, and serving costs by protecting salient weights during low-bit weight-only quantization [[17]]. Unlike the general architectural reduction seen in distillation, AWQ is specifically designed to accelerate token generation time across diverse deployment settings, ranging from edge devices like NVIDIA Jetson Orin to cloud-based GPU servers [[17]].

Recent frontier research has also explored more granular architectural interventions to increase speed during one-token generation. One such strategy involves skipping latter attention sublayers to avoid the computational expense of certain layers [[18]].

| Method | Optimization Layer | Primary Efficiency Goal | Target Deployment | | :— | :— | :— | :— | | DistilBERT [[15]] | Architecture | 60% faster inference vs BERT [[1]] | Constrained/Edge | | GPTQ [[16]] | Weights | Storage and multiplication reduction | Not specified | | AWQ [[17]] | Weights | Memory/bandwidth and token generation time | Edge to Cloud |

While distillation provides a path to significantly faster inference by shrinking the model itself [[15]], it is a heavy-duty process compared to the post-training weight optimizations offered by GPTQ and AWQ [[16], [17]]. Quantization methods like AWQ offer a more surgical approach to efficiency, optimizing for memory bandwidth and serving costs which are critical for modern LLM deployment [[17]], whereas distillation is more suited for creating entirely new, lightweight model variants [[14]].

Quantization techniques have evolved from simple 8-bit integer matrix multiplication ([19]) to more sophisticated methods like SmoothQuant, which balances quantization difficulty between weights and activations ([19]). Further refinements include group-wise quantization strategies like ZeroQuant ([20]) and SparseGPT, which enables massive one-shot pruning to achieve high sparsity without retraining ([21]).

1.4 Memory-Efficient Attention and Serving Systems

Research into making LLM inference more efficient operates at two distinct levels: the architectural design of the attention mechanism and the management of the serving engine. At the architectural level, FlashAttention ([22]) optimizes the attention module itself by employing tiling and recomputation. This approach reduces HBM reads/writes and intermediate matrix storage, providing wall-clock speedups and improved memory efficiency on GPUs. This architectural optimization serves as a foundational component for several engine-level systems, including DistServe ([23]) and PagedAttention ([24]).

Engine-level optimizations focus on maximizing resource utilization, specifically regarding the KV cache and the distinct computational phases of inference. PagedAttention ([24]) introduces a paging-inspired mechanism to manage KV cache using non-contiguous blocks, which enables fine-grained memory sharing and near-zero fragmentation. This method significantly improves throughput, achieving a 2–4$\times$ improvement over systems like FasterTransformer and Orca. However, a notable weakness is that the latency penalty introduced by the kernel's non-contiguous memory access patterns is not explicitly quantified ([24]).

The following table summarizes the primary characteristics of recent engine-level serving methods:

| Method | Optimization Layer | Deployment Setting | Primary Efficiency Goal | | :— | :— | :— | :— | | [24] | Engine | Cloud | High throughput serving | | [25] | Engine | Heterogeneous clusters | Cost, throughput, and power | | [23] | Engine | Physical cluster | Maximize per-GPU goodput | | [26] | Engine | Online inference serving | Throughput and latency (TBT) |

Recent advancements have moved toward managing the tension between the prefill and decoding phases. Splitwise ([25]) addresses this by separating the prompt computation and token generation phases onto separate machines within heterogeneous clusters to optimize for cost, throughput, and power. This concept of phase-based separation is extended by DistServe ([23]), which disaggregates the prefill and decoding phases and employs a novel placement algorithm to maximize per-GPU goodput in physical clusters. To further manage the inherent tension between throughput and latency, Sarathi-Serve ([26]) introduces chunked-prefills and a stall-free batching scheduler, utilizing hybrid batching to combine memory-bound decoding with compute-bound prefill tasks. Together,

these advancements illustrate a shift from low-level memory management toward high-level, phase-aware scheduling and disaggregation strategies designed to optimize the entire inference lifecycle across distributed hardware.

Memory-efficient attention mechanisms have seen significant speedups through improved parallelism and work partitioning in FlashAttention-2 ([27]). To further manage the KV cache, systems like SGLang leverage runtime reuse across multiple generation calls to accelerate structured language programs ([28]).

1.5 Optimizing Throughput via Speculative and Parallel Decoding

Current research into accelerating Large Language Model (LLM) inference focuses heavily on the decoding mechanism, specifically through speculative execution paradigms that decouple the generation of candidate tokens from their verification. The foundational approach, speculative decoding, utilizes a draft-verify-accept paradigm to improve walltime efficiency by predicting multiple tokens ahead of time [[29]]. This is further refined through speculative sampling, which employs a smaller, faster draft model and rejection sampling to ensure that the accelerated output maintains the statistical distribution of the target model, thereby reducing sampling latency [[30]].

While early methods focused on linear sequences of tokens, recent advancements have shifted toward more complex structural verification. SpecInfer introduces tree-based speculative inference, which moves beyond simple sequence matching to allow for tree-based parallel verification [[31]]. By verifying multiple potential token paths simultaneously within a tree structure, SpecInfer optimizes end-to-end latency in distributed and offloading-based deployment settings [[31]]. This represents a significant evolution from the basic speculative execution described in [[29]], moving the optimization layer from the decoding mechanism itself to the broader inference engine [[31]].

Recent frontier work has extended these principles to address multilingual requirements and specialized deployment contexts. For instance, researchers have integrated specialized drafters within the speculative decoding framework to optimize inference time in commercial settings [[32]]. This approach builds upon the foundational draft-verify-accept paradigm established in [[29]] and [[30]], while incorporating the structural efficiencies seen in tree-based methods [[32]].

| Method | Optimization Layer | Primary Efficiency Goal | Deployment Setting | | :— | :— | :— | :— | | [29] | Decoding mechanism | Walltime improvements | Single TPU-v4 | | [30] | Architecture (Draft + Target) | Sampling latency | Cloud (TPU v4) | | [31] | Engine | End-to-end latency | Distributed/Offloading | | [32] | Architecture | Inference time | Commercial settings |

A critical comparison of these methods reveals a progression in both complexity and deployment scope. Early implementations like [[29]] and [[30]] primarily address the latency of token generation and sampling within controlled environments like single or cloud-based TPUs. However, these methods are inherently limited by the linear nature of their verification. In contrast, the tree-based approach in [[31]] provides a more robust solution for distributed environments by allowing for parallel verification of multiple token branches, which directly addresses the bottleneck of end-to-end latency in complex serving scenarios. While the specialized drafter approach in [[32]] focuses on the architectural nuances of multilingualism, it relies on the same underlying draft-verify-accept logic, suggesting that the field is moving toward highly specialized assistant models to maximize the throughput gains originally proposed in [[29]].

1.6 Advanced Scheduling and Disaggregated Inference Architectures

Recent advancements in LLM inference efficiency have shifted from single-operator optimizations toward sophisticated engine-level scheduling and architectural disaggregation to manage the competing demands of throughput, latency, and memory. These approaches can be categorized by their deployment scale, ranging from single-GPU memory management to large-scale cloud-based model cascading and disaggregated execution.

At the cloud scale, researchers are addressing the interference between prefill and decode phases. TetriInfer [[33]] proposes a disaggregated architecture that separates prefill and decode instances, utilizing a two-level scheduling mechanism to optimize Time-to-First-Token (TTFT), Job Completion Time (JCT), and performance per dollar. This architectural split aims to mitigate the resource contention inherent in mixed downstream workloads [[33]]. Complementing this structural approach, Cascadia [[34]] optimizes cloud serving through a model cascade strategy. Instead of just managing phase interference, Cascadia routes queries between small, medium, and large models to minimize p95 latency and maximize throughput [[34]]. While TetriInfer focuses on the temporal separation of compute phases, Cascadia focuses on the complexity of the model itself to achieve efficiency.

A significant subset of research focuses on the intelligence of the scheduler itself. Rather than relying on static heuristics, recent work explores predictive scheduling. For instance, [[35]] introduces a Learning to Rank (LTR) approach for request scheduling, which predicts the relative ranks of output lengths to approximate Shortest Job First (SJF) or Shortest Remaining Time First (SRTF) policies, thereby improving latency and throughput on DGX-class hardware [[35]]. This predictive paradigm is further refined by the (Nested) WAIT algorithm [[36]], which extends the LTR approach [[35]] to focus on multi-stage online scheduling under strict memory constraints. Unlike the cloud-scale disaggregation of TetriInfer, WAIT is specifically designed to maximize the utilization of a single GPU by managing KV cache memory constraints [[36]].

| Method | Primary Optimization Layer | Deployment Setting | Primary Efficiency Goal | | :— | :— | :— | :— | | [33] | Engine (Disaggregation) | Cloud | TTFT, JCT, and perf/$ | | [35] | Engine (LTR Scheduling) | Cloud (8x A100) | Latency and throughput | | [36] | Engine (Multi-stage) | Single GPU (A100) | Single GPU utilization | | [34] | Engine (Model Cascade) | Cloud (32x† H100) | p95 latency and throughput |

The literature reveals a tension between architectural complexity and scheduling intelligence. Disaggregated architectures like TetriInfer [[33]] provide a robust foundation for handling diverse workloads but require complex infrastructure to manage the split between prefill and decode instances. In contrast, LTR-based scheduling [[35]] and WAIT [[36]] offer more granular, data-driven control over request ordering, which is particularly effective for maximizing hardware utilization or managing memory-constrained environments. Finally, Cascadia [[34]] represents a higher-level abstraction, where efficiency is gained not just by how tasks are scheduled, but by which model is selected to perform the task.

Serving engines are increasingly focused on maximizing throughput by managing GPU bubbles and compute utilization, as seen in SARATHI's chunked-prefill approach ([37]). Advanced scheduling strategies, such as predicting response lengths for micro-batching ([38]) or using MLFQ-based preemptive scheduling ([39]), further optimize job completion times in distributed environments. For resource-constrained settings, FlexGen demonstrates how high-throughput inference can be achieved on a single commodity GPU by optimizing I/O and memory costs ([40]).

1.7 The Frontier: Reasoning-Aware and Cost-Efficient Inference

Current research into large language model (LLM) efficiency is shifting from general compression toward specialized methods that preserve or enhance reasoning capabilities while minimizing computational overhead. These approaches vary significantly in their optimization layer, ranging from fundamental training adjustments to high-level model cascading.

| Method | Optimization Layer | Efficiency Goal | Technical Approach | | :— | :— | :— | :— | | [41] | Training | Token-efficiency | Length-penalized RL | | [42] | Architecture | Inference time | Knowledge distillation | | [43] | Architecture/Algorithm | Throughput/Cost | Early-exiting & parallel decoding | | [44] | Architecture | Latency | Speculative decoding | | [45] | Model Selection | Cost/Tokens | Confidence-driven cascading |

Optimization strategies can be categorized by whether they modify the model's internal mechanics or manage how models are deployed. At the training layer, [41] employs reinforcement learning (RL) with a length-penalizing objective function to minimize inference-time compute by reducing unnecessary token generation. This differs from architectural-level optimizations, such as [42], which focuses on distilling reasoning capabilities into more efficient architectures (e.g., Mamba) to achieve a 2.5x speedup in inference time.

Other researchers focus on algorithmic co-design to accelerate execution. [43] seeks to improve throughput and computational costs through a combination of early-exiting algorithms, parallel decoding, and deep parameter sharing. In contrast, [44] targets latency specifically through inference-cost-aware dynamic tree construction, utilizing speculative decoding to optimize GPU deployment.

A distinct paradigm is the use of model cascading to manage costs across different model scales. [45] introduces a confidence-driven dynamic selection framework that uses a dual-metric strategy—combining token probability $P(T)$ with a trained MLP classifier $P(IK)$ based on hidden states—to escalate queries through a hierarchy of LLMs. This approach demonstrates significant practical utility, achieving a 60% reduction in GPT-4o token usage and a 36.46% reduction in computational cost when scaling from 8B to 70B LLaMA models, while maintaining accuracy. However, the method's reliance on a supervised $P(IK)$ classifier introduces a dependency on specific datasets, and the multi-stage escalation process presents risks regarding increased end-to-end latency and potential error propagation in long reasoning chains. While [46] also explores enhanced reasoning efficiency, the aforementioned methods represent the primary divergence between modifying a single model's internal efficiency and managing a heterogeneous ensemble of models.

1.8 Further Reading

- [47] (2022) — "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale" [core, cited-by 73 in-topic]

1.9 Tensions and Open Problems

A primary tension in the literature concerns the optimal strategy for balancing model precision with computational throughput. On one hand, quantization-centric approaches emphasize maintaining high fidelity through sophisticated bit-width management, such as the 8-bit matrix multiplication techniques proposed in [47]. However, this focus on precision often conflicts with the need for extreme throughput in resource-constrained environments. For instance, while [19] and [20] suggest

that smoothing activations or using zero-point quantization can preserve accuracy, these methods may not scale as effectively as hardware-aware offloading strategies. This is exemplified by [40], which argues that high-throughput generative inference is better achieved through aggressive memory management and offloading rather than solely relying on the precision-preserving quantization schemes found in [47] or the specialized inference engines described in [48].

Another significant debate centers on the architectural trade-offs between model scale and specialized decoding efficiency. One school of thought focuses on the inherent power of massive, open-access multilingual models like those in [5] or the dense scaling seen in [49]. In contrast, recent work suggests that the path to efficiency lies in "speculative" architectures. While [29] introduced the foundational concept of using a smaller draft model to accelerate a larger target model, there is ongoing disagreement regarding the optimal configuration of these "big-little" pairs. For example, [50] suggests that more sophisticated, learned draft models can outperform the simpler speculative approaches described in [30], yet the overhead of managing these auxiliary models remains a point of contention regarding their net latency benefits.

Finally, there is a growing tension between structural pruning and parameter-efficient fine-tuning (PEFT) for model adaptation. While methods like LoRA in [8] focus on adding low-rank adapters to keep the original weights frozen, structural pruning approaches, such as those in [51], attempt to physically remove redundant parameters to reduce the model's footprint. This creates a fundamental disagreement: should we optimize for "thin" models that are inherently efficient, or should we focus on "modular" models that can be rapidly adapted via lightweight layers? This debate is further complicated by the emergence of highly efficient foundation models like LLaMA in [52], which challenge the necessity of the massive-scale pre-training paradigms used in earlier models like BLOOM ([5]).

1.10 Outlook

The frontier of large language model research is rapidly shifting from general scaling toward specialized, cost-aware deployment. As the field moves toward 2026, we see a transition from monolithic inference to decentralized and decomposed architectures, such as the parallelized inference techniques in [53] and the multi-agent frameworks in [54]. The next generation of breakthroughs will likely emerge at the intersection of reasoning-intensive computation and edge-based deployment, where models must not only be compact—as explored in [55]—but also capable of verifiable, privacy-preserving reasoning in resource-constrained environments ([56]). Ultimately, the goal is to move beyond mere parameter reduction toward "intelligence density," where the cost-per-reasoning-step becomes the primary metric of success.

— *† number could not be verified against the cited paper's abstract or extracted fields (figure- or full-text-sourced).*

Efficient Memory Management for Large Language Model Serving with PagedAttention — [24] p.1

ment for Large Language
h *PagedAttention*

Ying Sheng[1,2] Lianmin Zheng[1] Cody Hao Yu[3]
ao Zhang[4] Ion Stoica[1]
dependent Researcher [4]UC San Diego

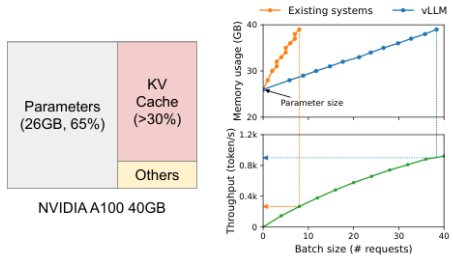


Figure 1. *Left:* Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request. A small amount of memory (yellow) is used ephemerally for activation. *Right:* vLLM smooths out the rapid growth curve of KV cache memory seen in existing systems [31, 60], leading to a notable boost in serving throughput.

The left panel shows a rectangular memory layout for an NVIDIA A100 40GB GPU, divided into three labeled boxes: "Parameters" (26GB, 65% in gray), "KV Cache" (>30% in red), and "Others" (in yellow). The right panel contains two line graphs with axes for "Memory usage (GB)" and "Throughput (token/s)" against "Batch size (# requests)," comparing "Existing systems" (orange) to "vLLM" (blue). The figure conveys that vLLM manages memory more efficiently by smoothing the rapid growth of KV cache memory usage, resulting in a significant boost in throughput compared to existing systems.

DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving — [23] p.1

[3] Jianbo Hu[1] Yibo Zhu[2] Xuanzhe Liu[1]
ao Zhang[3]

[1]niversity [2]StepFun [3]UC San Diego

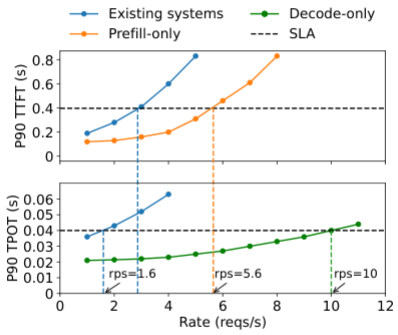


Figure 1: Performance when serving an LLM with 13B parameters under a synthetic workload with input length = 512 and output length = 64 on one NVIDIA 80GB A100. *Upper*: The P90 time-to-first-token (TTFT) latency comparing existing systems vs. a system serving only the prefill phase. *Down*: The P90 time-per-output-token (TPOT) latency comparing existing systems vs. a system serving only the decoding phase.

The figure consists of two vertically stacked line graphs sharing a common x-axis labeled "Rate (reqs/s)." The upper graph plots P90 TTFT (s) against the request rate, comparing "Existing systems" (blue), "Prefill-only" (orange), and "Decode-only" (green) curves against a horizontal dashed SLA line. The lower graph plots P90 TPOT (s) against the request rate, showing the same three system types and the SLA line, illustrating how latency scales with throughput for each serving configuration.

Splitwise: Efficient generative LLM inference using phase splitting — [25] p.2

and H100 GPUs using production traces.

2) Splitwise, our technique for optimized utilization of available hardware, which splits the prompt computation and token generation phases onto separate machines.
3) A design exploration of homogeneous and heterogeneous cluster deployments with Splitwise to optimize the overall cost, request throughput, and provisioned power.
4) An evaluation of the systems designed with Splitwise using production traces.

II. Background

A. Large Language Models

Modern LLMs are based on transformers. Transformer models use attention [77] and multi-layer-perceptron layers to understand the inputs and generate an output, respectively. Transformer-based LLMs include encoder-only [36], [54], decoder-only [67], [69], [71], and encoder-decoder [70] models. Generative LLMs, the focus of this paper, are usually either decoder-only, or encoder-decoder models.

B. Generative LLM inference phases

Figure 1 shows an example of generative LLM inference. Once the prompt query is received, all the input tokens are computed in parallel, within a single iteration, to generate the first token. We call this the prompt processing phase. The context generated from the attention layers during the prompt computation is saved in the key-value (KV) cache, since it

The figure illustrates the two distinct phases of generative LLM inference: the prompt processing phase and the token generation phase. It uses a sequence of boxes and arrows to depict how input tokens are processed in parallel to populate a key-value (KV) cache, followed by the autoregressive generation of subsequent tokens.

FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU — [40] p.1

e Inference of Large Language Models
ıgle GPU

an Li [2] **Max Ryabinin** [4 5] **Daniel Y. Fu** [1] **Zhiqiang Xie** [1]
ɛrcy Liang [1] **Christopher Ré** [1] **Ion Stoica** [2] **Ce Zhang** [3]

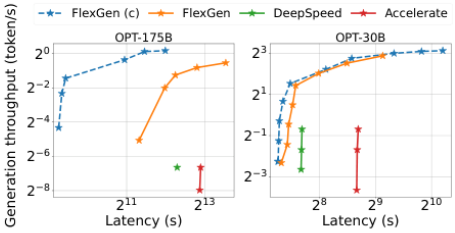


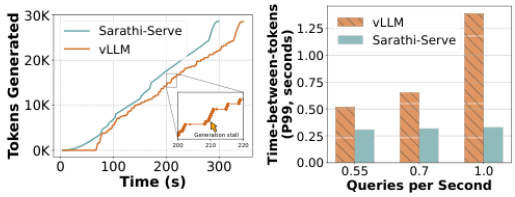
Figure 1. The total latency for a block and throughput trade-offs of three offloading-based systems for OPT-175B (left) and OPT-30B (right) on a single NVIDIA T4 (16 GB) GPU with 208 GB CPU DRAM and 1.5TB SSD. FlexGen achieves a new Pareto-optimal frontier with 100× higher maximum throughput for OPT-175B. Other systems cannot further increase throughput due to out-of-memory issues. "(c)" denotes compression.

The figure consists of two line graphs plotting generation throughput (tokens/s) against latency (s) for the OPT-175B (left) and OPT-30B (right) models. Each plot compares four systems—FlexGen (c), FlexGen, DeepSpeed, and Accelerate—using colored lines and markers to demonstrate that FlexGen achieves a higher throughput at various latency levels compared to the other baselines.

Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve — [26] p.1

ıwar[1], Jayashree Mohan[1], Nipun Kwatra[1],
ınov[2], and Ramachandran Ramjee[1]

:search India
ɛ of Technology



(a) Generation stall. (b) High tail latency.

Figure 1: Yi-34B running on two A100 GPUs serving 128 requests from *arxiv-summarisation* trace. 1a highlights one of the many generation stalls lasting over several seconds in vLLM [53]. 1b shows the impact of increasing load on tail latency. Sarathi-Serve improves throughput while eliminating generation stalls.

Figure 1a is a line graph plotting "Tokens Generated" against "Time (s)" for two methods, Sarathi-Serve and vLLM, featuring an inset box highlighting a "Generation stall" in the vLLM trace. Figure 1b is a bar chart showing "Time-between-tokens (p99, seconds)" on the y-axis against "Queries per Second" on the x-axis, comparing vLLM and Sarathi-Serve across three load levels. Together, the figures convey that Sarathi-Serve maintains lower tail latency and avoids the significant generation stalls observed in vLLM as request load increases.

FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning — [27] p.4

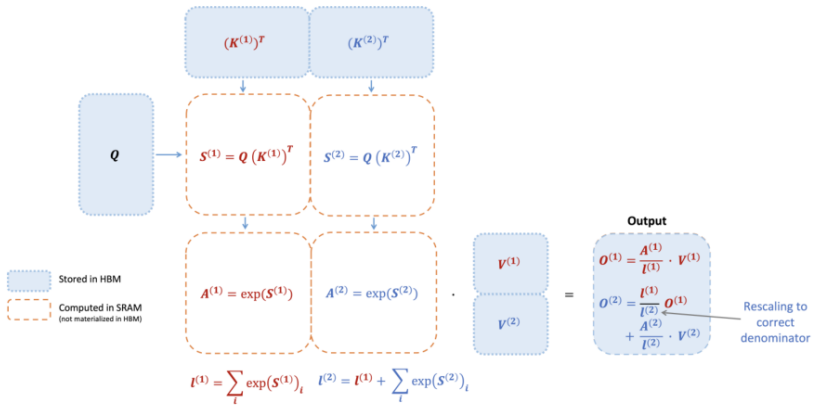


Figure 1: Diagram of how FlashAttention forward pass is performed, when the key **K** is partitioned into two blocks and the value **V** is also partitioned into two blocks. By computing attention with respect to each block and rescaling the output, we get the right answer at the end, while avoiding expensive memory reads/writes of the intermediate matrices **S** and **P**. We simplify the diagram, omitting the step in softmax that subtracts each element by the row-wise max.

The figure uses blue shaded boxes to represent data stored in HBM (Q, K, V) and orange dashed

boxes to represent intermediate computations performed in SRAM (S, A, and l). Arrows illustrate the flow of operations where Q is multiplied by partitioned blocks of K^T to produce S, which is then exponentiated to A and used to compute partial outputs $O^{(1)}$ and $O^{(2)}$. The diagram conveys how the final output $O^{(2)}$ is reconstructed by rescaling the partial outputs using the accumulated sums of exponents ($l^{(1)}$ and $l^{(2)}$) to ensure a correct denominator.

AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving — [13] p.1

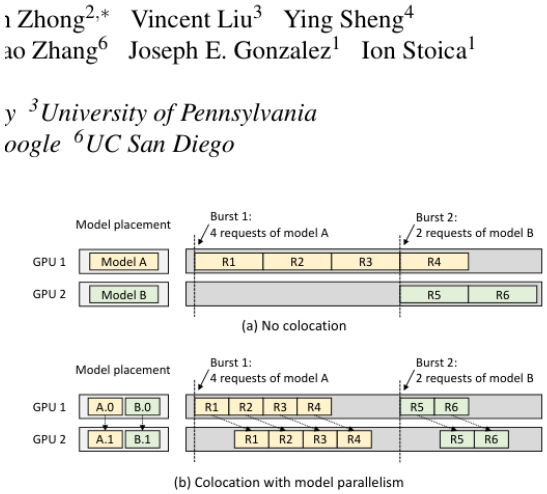


Figure 1: Two placement strategies for serving two models on two GPUs. In each subfigure, the left part shows the model placements and the right part shows the timeline for handling bursty requests. At the time of "Burst 1", 4 requests of model A come at the same time. Colocation with model parallelism can reduce the average completion time of bursty requests.

The figure compares two placement strategies, "(a) No colocation" and "(b) Colocation with model parallelism," using horizontal bars to represent GPU 1 and GPU 2. In each subfigure, the left side shows model placement (single models in (a) versus partitioned models in (b)), while the right side uses labeled boxes (R1–R6) and arrows to illustrate how bursty requests are processed over time. The figure conveys that colocation with model parallelism allows for more efficient request handling, reducing the completion time for bursty requests compared to no colocation.

Fast Distributed Inference Serving for Large Language Models — [39] p.1

ɪr Large Language Models

Zhang* Shengyu Liu
ıg Xuanzhe Liu Xin Jin
ity

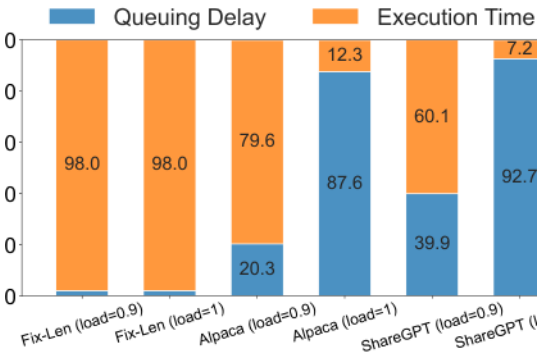


Figure 1: Head-of-line blocking in LLM inference.

This bar chart compares the components of latency—Queuing Delay (blue) and Execution Time (orange)—across different LLM serving scenarios. It illustrates "head-of-line blocking" by showing that as load increases from 0.9 to 1.0, the queuing delay significantly increases while execution time remains relatively stable for models like Alpaca and ShareGPT.

AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration — [17] p.1

ıı error. To identify salient weight channels, we should
ıid the hardware-inefficient mix-precision quantization,
ınnels can reduce the quantization error. AWQ employs
t channels to protect them. The scale is determined by
not rely on any backpropagation or reconstruction, so
ıout overfitting the calibration set. AWQ outperforms
ıin-specific benchmarks (coding and math). Thanks to
ɔerformance for *instruction-tuned* LMs and, for the first
TinyChat, an efficient and flexible inference framework
ion and platform-aware weight packing, TinyChat offers
ılementation on both desktop and mobile GPUs. It also
on mobile GPUs.

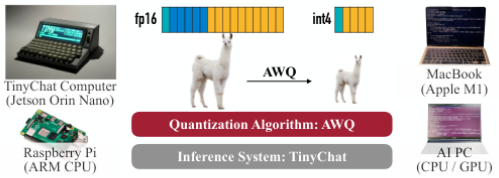


Figure 1. We introduce **AWQ**, a versatile weight quantization method for LLM. To implement AWQ, we developed **TinyChat** to deploy 4-bit quantized LLMs into various edge platforms, achieving a **3-4**× performance boost compared to FP16. Notably, we've also manufactured a **TinyChat computer**, powered by TinyChat, which contains an NVIDIA Jetson Orin Nano with only 8GB of memory and 15W power consumption. Demo: https://youtu.be/z91a8DrfgEw.

Figure 1 illustrates the AWQ quantization algorithm and the TinyChat inference system, showing how AWQ protects salient weight channels to enable efficient LLM deployment. The diagram uses arrows to connect the AWQ process to various hardware platforms, including a TinyChat computer (Jetson Orin Nano), a Raspberry Pi (ARM CPU), and a MacBook (Apple M1), as well as a general AI PC. The figure conveys that the combination of AWQ and TinyChat allows for 4-bit quantized LLMs to achieve a 3-4× performance boost across diverse desktop and mobile hardware.

SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills — [37] p.1

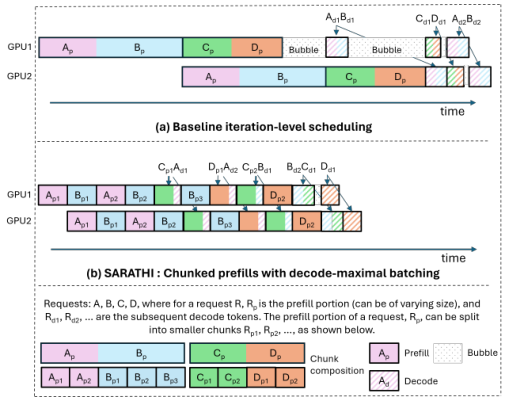


Figure 1: **Example two-stage pipeline parallel schedule.** (a) In prior solutions like Orca [48], pipeline bubbles are common due to varying prompt and decode compute times. Further, decodes are highly inefficient (decode *cost-per-token* is order-of-magnitude higher than Prefill). (b) SARATHI significantly reduces pipeline bubbles and enables more efficient *piggybacked decodes*.

The figure compares two scheduling approaches for LLM inference across two GPUs, using colored blocks to represent request chunks (A_p, B_p, C_p, D_p) and white blocks to represent idle "bubbles." Panel (a) illustrates baseline iteration-level scheduling where large bubbles occur due to the sequential nature of prompt prefill and token decoding, while panel (b) shows the SARATHI method where chunked prefills are interleaved with decodes to minimize idle time.

Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads — [33] p.2

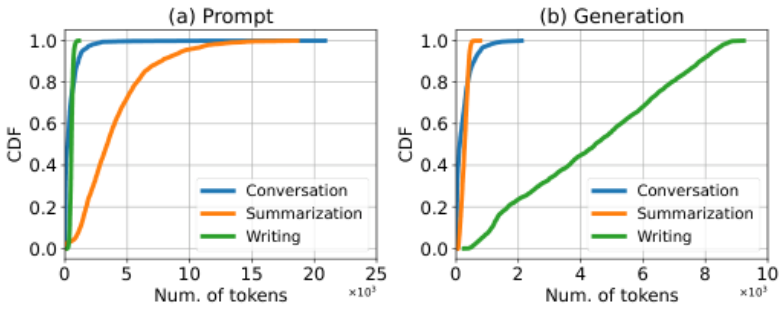


Figure 1: **Length Distribution.** Prompt Tokens for Prefill and Generated Tokens during Decode. Data sources: conversation [35], summarization [17], writing [18].

The figure consists of two side-by-side line graphs, (a) Prompt and (b) Generation, plotting the Cumulative Distribution Function (CDF) against the number of tokens on the x-axes. Each plot contains three colored lines representing different workloads—Conversation (blue), Summarization (orange), and Writing (green)—to show the distribution of prompt tokens for prefill and generated tokens for decode. The graphs convey that prompt lengths vary significantly across tasks, while generation lengths show distinct, task-specific distributions, with writing tasks typically producing the highest number of tokens.

SGLang: Efficient Execution of Structured Language Model Programs — [28] p.2

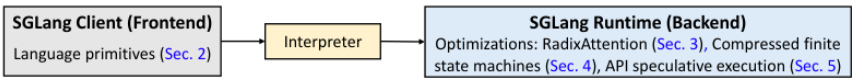
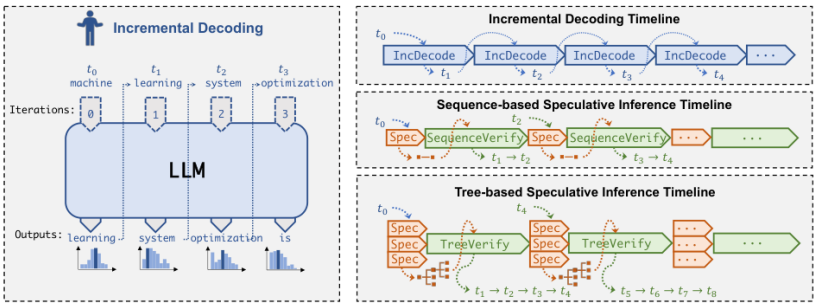


Figure 1: System architecture: An interpreter executes language primitives with optimized runtime.

The figure depicts a system architecture diagram consisting of three sequential boxes connected by arrows: an "SGLang Client (Frontend)" containing language primitives, an "Interpreter," and an "SGLang Runtime (Backend)." The backend box lists specific optimizations including RadixAttention, compressed finite state machines, and API speculative execution. The diagram conveys the flow of execution from the frontend through an interpreter to an optimized runtime backend.

SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification — [31] p.3



(a) Incremental decoding. **(b)** Timeline Comparison.

Figure 1. Comparing the incremental decoding approach used by existing LLM serving systems, the sequence-based speculative inference approach, and the tree-based speculative inference approach used by SpecInfer.

Figure 1 compares three LLM serving methods through timelines and process diagrams. Panel (a) illustrates incremental decoding as a sequential process where an LLM generates one token at a time (e.g., "learning," "system," "optimization") across discrete iterations. Panel (b) uses colored boxes and arrows to contrast the timelines of incremental decoding (one token per step), sequence-based speculative inference (a block of speculative tokens followed by verification), and the proposed tree-based speculative inference (multiple speculative paths verified in a single step).

Response Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline — [38] p.2

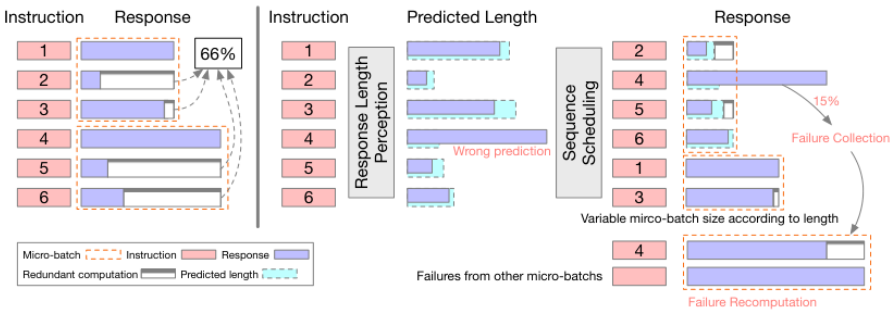


Figure 1: **Left:** Vanilla batch inference leads to underperformance with 66% redundant tokens when short and long responses are in the same batch. **Right:** The pipeline of our sequence scheduling. First, the response length perception module estimates the response length for each instruction. Sequence scheduling groups instructions with similar predicted lengths together and larger batch sizes for shorter responses. Failure collection and recomputation strategy is adopted to avoid wrong predictions degenerating the performance.

The figure consists of two panels comparing vanilla batch inference with the proposed pipeline. The left panel shows a micro-batch of six instructions with varying response lengths, highlighting that 66% of tokens are redundant due to the mismatch between short and long responses. The right panel illustrates the proposed pipeline, where a "Response Length Perception" module estimates

lengths, a "Sequence Scheduling" module groups instructions into variable micro-batches based on those lengths, and a "Failure Collection" and "Failure Recomputation" mechanism handles errors from wrong predictions.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. `https://arxiv.org/abs/1706.03762`, 2017.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, , and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. `https://arxiv.org/abs/1810.04805`, 2018.

[3] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. `https://arxiv.org/abs/1910.10683`, 2019.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. `https://arxiv.org/abs/2005.14165`, 2020.

[5] BigScience Workshop: Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel van Strien, David Ifeoluwa Adelani, Dragomir Radev, Eduardo González Ponferrada, Efrat Levkovizh, Ethan Kim, Eyal Bar Natan, Francesco De Toni, Gérard Dupont, Germán Kruszewski, Giada Pistilli, Hady Elsahar, Hamza Benyamina, Hieu Tran, Ian Yu, Idris Abdulmumin, Isaac Johnson, Itziar Gonzalez-Dios, Javier de la Rosa, Jenny Chim, Jesse Dodge, Jian Zhu, Jonathan Chang, Jörg Frohberg, Joseph Tobing, Joydeep Bhattacharjee, Khalid Almubarak, Kimbo Chen, Kyle Lo, Leandro Von Werra, Leon Weber, Long Phan, Loubna Ben allal, Ludovic Tanguy, Manan Dey, Manuel Romero Mu noz, Maraim Masoud, María Grandury, Mario Šaško, Max Huang, Maximin Coavoux, Mayank Singh, Mike Tian-Jian Jiang, Minh Chien Vu, Mohammad A. Jauhar, Mustafa Ghaleb, Nishant Subramani, Nora Kassner, Nurulaqilla Khamis, Olivier Nguyen, Omar Espejel, Ona de Gibert, Paulo Villegas, Peter Henderson, Pierre Colombo, Priscilla Amuok, Quentin Lhoest, Rheza Harliman, Rishi Bommasani, Roberto Luis López, Rui Ribeiro, Salomey Osei, Sampo Pyysalo, Sebastian Nagel, Shamik Bose, Shamsuddeen Hassan Muhammad, Shanya Sharma, Shayne Longpre, Somaieh Nikpoor, Stanislav Silberberg, Suhas Pai, Sydney Zink, Tiago Timponi Torrent, Timo Schick, Tristan Thrush, Valentin Danchev, Vassilina Nikoulina, Veronika Laippala,

Violette Lepercq, Vrinda Prabhu, Zaid Alyafeai, Zeerak Talat, Arun Raja, Benjamin Heinzerling, Chenglei Si, Davut Emre Taşar, Elizabeth Salesky, Sabrina J. Mielke, Wilson Y. Lee, Abheesht Sharma, Andrea Santilli, Antoine Chaffin, Arnaud Stiegler, Debajyoti Datta, Eliza Szczechla, Gunjan Chhablani, Han Wang, Harshit Pandey, Hendrik Strobelt, Jason Alan Fries, Jos Rozen, Leo Gao, Lintang Sutawika, M Saiful Bari, Maged S. Al-shaibani, Matteo Manica, Nihal Nayak, Ryan Teehan, Samuel Albanie, Sheng Shen, Srulik Ben-David, Stephen H. Bach, Taewoon Kim, Tali Bers, Thibault Fevry, Trishala Neeraj, Urmish Thakker, Vikas Raunak, Xiangru Tang, Zheng-Xin Yong, Zhiqing Sun, Shaked Brody, Yallow Uri, Hadar Tojarieh, Adam Roberts, Hyung Won Chung, Jaesung Tae, Jason Phang, Ofir Press, Conglong Li, Deepak Narayanan, Hatim Bourfoune, Jared Casper, Jeff Rasley, Max Ryabinin, Mayank Mishra, Minjia Zhang, Mohammad Shoeybi, Myriam Peyrounette, Nicolas Patry, Nouamane Tazi, Omar Sanseviero, Patrick von Platen, Pierre Cornette, Pierre François Lavallée, Rémi Lacroix, Samyam Rajbhandari, Sanchit Gandhi, Shaden Smith, Stéphane Requena, Suraj Patil, Tim Dettmers, Ahmed Baruwa, Amanpreet Singh, Anastasia Cheveleva, Anne-Laure Ligozat, Arjun Subramonian, Aurélie Névéol, Charles Lovering, Dan Garrette, Deepak Tunuguntla, Ehud Reiter, Ekaterina Taktasheva, Ekaterina Voloshina, Eli Bogdanov, Genta Indra Winata, Hailey Schoelkopf, Jan-Christoph Kalo, Jekaterina Novikova, Jessica Zosa Forde, Jordan Clive, Jungo Kasai, Ken Kawamura, Liam Hazan, Marine Carpuat, Miruna Clinciu, Najoung Kim, Newton Cheng, Oleg Serikov, Omer Antverg, Oskar van der Wal, Rui Zhang, Ruochen Zhang, Sebastian Gehrmann, Shachar Mirkin, Shani Pais, Tatiana Shavrina, Thomas Scialom, Tian Yun, Tomasz Limisiewicz, Verena Rieser, Vitaly Protasov, Vladislav Mikhailov, Yada Pruksachatkun, Yonatan Belinkov, Zachary Bamberger, Zdeněk Kasner, Alice Rueda, Amanda Pestana, Amir Feizpour, Ammar Khan, Amy Faranak, Ana Santos, Anthony Hevia, Antigona Unldreaj, Arash Aghagol, Arezoo Abdollahi, Aycha Tammour, Azadeh HajiHosseini, Bahareh Behroozi, Benjamin Ajibade, Bharat Saxena, Carlos Mu noz Ferrandis, Daniel McDuff, Danish Contractor, David Lansky, Davis David, Douwe Kiela, Duong A. Nguyen, Edward Tan, Emi Baylor, Ezinwanne Ozoani, Fatima Mirza, Frankline Ononiwu, Habib Rezanejad, Hessie Jones, Indrani Bhattacharya, Irene Solaiman, Irina Sedenko, Isar Nejadgholi, Jesse Passmore, Josh Seltzer, Julio Bonis Sanz, Livia Dutra, Mairon Samagaio, Maraim Elbadri, Margot Mieskes, Marissa Gerchick, Martha Akinlolu, Michael McKenna, Mike Qiu, Muhammed Ghauri, Mykola Burynok, Nafis Abrar, Nazneen Rajani, Nour Elkott, Nour Fahmy, Olanrewaju Samuel, Ran An, Rasmus Kromann, Ryan Hao, Samira Alizadeh, Sarmad Shubber, Silas Wang, Sourav Roy, Sylvain Viguier, Thanh Le, Tobi Oyebade, Trieu Le, Yoyo Yang, Zach Nguyen, Abhinav Ramesh Kashyap, Alfredo Palasciano, Alison Callahan, Anima Shukla, Antonio Miranda-Escalada, Ayush Singh, Benjamin Beilharz, Bo Wang, Caio Brito, Chenxi Zhou, Chirag Jain, Chuxin Xu, Clémentine Fourrier, Daniel León Peri nán, Daniel Molano, Dian Yu, Enrique Manjavacas, Fabio Barth, Florian Fuhrimann, Gabriel Altay, Giyaseddin Bayrak, Gully Burns, Helena U. Vrabec, Imane Bello, Ishani Dash, Jihyun Kang, John Giorgi, Jonas Golde, Jose David Posada, Karthik Rangasai Sivaraman, Lokesh Bulchandani, Lu Liu, Luisa Shinzato, Madeleine Hahn de Bykhovetz, Maiko Takeuchi, Marc Pàmies, Maria A Castillo, Marianna Nezhurina, Mario Sänger, Matthias Samwald, Michael Cullan, Michael Weinberg, Michiel De Wolf, Mina Mihaljcic, Minna Liu, Moritz Freidank, Myungsun Kang, Natasha Seelam, Nathan Dahlberg, Nicholas Michio Broad, Nikolaus Muellner, Pascale Fung, Patrick Haller, Ramya Chandrasekhar, Renata Eisenberg, Robert Martin, Rodrigo Canalli, Rosaline Su, Ruisi Su, Samuel Cahyawijaya, Samuele Garda, Shlok S Deshmukh, Shubhanshu Mishra, Sid Kiblawi, Simon Ott, Sinee Sang-aroonsiri, Srishti Kumar, Stefan Schweter, Sushil Bharati, Tanmay Laud, Théo Gigant, Tomoya Kainuma, Wojciech Kusa, Yanis Labrak, Yash Shailesh Bajaj, Yash Venkatraman, Yifan Xu, Yingxin Xu, Yu Xu, Zhe Tan, Zhongli Xie, Zifan Ye,

Mathilde Bras, Younes Belkada, and Thomas Wolf. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. https://arxiv.org/abs/2211.05100, 2022.

[6] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. https://arxiv.org/abs/2104.04473, 2021.

[7] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. https://arxiv.org/abs/2305.13245, 2023.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685, 2021.

[9] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. https://arxiv.org/abs/1811.06965, 2018.

[10] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, , and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. https://arxiv.org/abs/1909.08053, 2019.

[11] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models. https://arxiv.org/abs/2102.07988, 2021.

[12] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. https://arxiv.org/abs/2201.12023, 2022.

[13] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. https://arxiv.org/abs/2302.11665, 2023.

[14] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. https://arxiv.org/abs/1503.02531, 2015.

[15] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. https://arxiv.org/abs/1910.01108, 2019.

[16] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. https://arxiv.org/abs/2210.17323, 2022.

[17] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. https://arxiv.org/abs/2306.00978, 2023.

[18] Georgy Tyukin. Enhancing Inference Efficiency of Large Language Models: Investigating Optimization Strategies and Architectural Innovations. https://arxiv.org/abs/2404.05741, 2024.

[19] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. https://arxiv.org/abs/2211.10438, 2022.

[20] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. https://arxiv.org/abs/2206.01861, 2022.

[21] Elias Frantar and Dan Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. https://arxiv.org/abs/2301.00774, 2023.

[22] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. https://arxiv.org/abs/2205.14135, 2022.

[23] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. https://arxiv.org/abs/2401.09670, 2024.

[24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. https://arxiv.org/abs/2309.06180, 2023.

[25] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Í nigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. https://arxiv.org/abs/2311.18677, 2023.

[26] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. https://arxiv.org/abs/2403.02310, 2024.

[27] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. https://arxiv.org/abs/2307.08691, 2023.

[28] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. https://arxiv.org/abs/2312.07104, 2023.

[29] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. https://arxiv.org/abs/2211.17192, 2022.

[30] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. https://arxiv.org/abs/2302.01318, 2023.

[31] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen,

Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification. https://arxiv.org/abs/2305.09781, 2023.

[32] Euiin Yi, Taehyeon Kim, Hongseok Jeung, Du-Seong Chang, and Se-Young Yun. Towards Fast Multilingual LLM Inference: Speculative Decoding and Specialized Drafters. https://arxiv.org/abs/2406.16758, 2024.

[33] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. https://arxiv.org/abs/2401.11181, 2024.

[34] Youhe Jiang, Fangcheng Fu, Wanru Zhao, Stephan Rabanser, Jintao Zhang, Nicholas D. Lane, and Binhang Yuan. Cascadia: An Efficient Cascade Serving System for Large Language Models. https://arxiv.org/abs/2506.04203, 2025.

[35] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient LLM Scheduling by Learning to Rank. https://arxiv.org/abs/2408.15792, 2024.

[36] Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints. https://arxiv.org/abs/2504.11320, 2025.

[37] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. https://arxiv.org/abs/2308.16369, 2023.

[38] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline. https://arxiv.org/abs/2305.13144, 2023.

[39] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast Distributed Inference Serving for Large Language Models. https://arxiv.org/abs/2305.05920, 2023.

[40] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. https://arxiv.org/abs/2303.06865, 2023.

[41] Daman Arora and Andrea Zanette. Training Language Models to Reason Efficiently. https://arxiv.org/abs/2502.04463, 2025.

[42] Daniele Paliotta, Junxiong Wang, Matteo Pagliardini, Kevin Y. Li, Aviv Bick, J. Zico Kolter, Albert Gu, François Fleuret, and Tri Dao. Thinking Slow, Fast: Scaling Inference Compute with Distilled Reasoners. https://arxiv.org/abs/2502.20339, 2025.

[43] Sangmin Bae. Accelerating Large Language Model Inference via Early-Exiting Algorithms. https://arxiv.org/abs/2509.05915, 2025.

[44] Yinrong Hong, Zhiquan Tan, and Kai Hu. Inference-Cost-Aware Dynamic Tree Construction for Efficient Inference in Large Language Models. https://arxiv.org/abs/2510.26577, 2025.

[45] Bo-Wei Chen, Chung-Chi Chen, and An-Zi Yen. Confidence-Driven Multi-Scale Model Selection for Cost-Efficient Inference. https://arxiv.org/abs/2602.22090, 2026.

[46] Leslie G. Valiant. Enhanced and Efficient Reasoning in Large Learning Models. https://arxiv.org/abs/2605.14036, 2026.

[47] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. https://arxiv.org/abs/2208.07339, 2022.

[48] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. https://arxiv.org/abs/2207.00032, 2022.

[49] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. OPT: Open Pre-trained Transformer Language Models. https://arxiv.org/abs/2205.01068, 2022.

[50] Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. Speculative Decoding with Big Little Decoder. https://arxiv.org/abs/2302.07863, 2023.

[51] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the Structural Pruning of Large Language Models. https://arxiv.org/abs/2305.11627, 2023.

[52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models. https://arxiv.org/abs/2302.13971, 2023.

[53] You-Liang Huang, Xinhao Huang, Chengxi Liao, and Zeyi Wen. DeInfer: Efficient Parallel Inferencing for Decomposed Large Language Models. https://arxiv.org/abs/2604.17709, 2026.

[54] Sixun Dong, Juhua Hu, Steven Li, Wei Wen, and Qi Qian. Rethinking Model Efficiency: Multi-Agent Inference with Large Models. https://arxiv.org/abs/2604.04929, 2026.

[55] Dari Baturova, Elena Bruches, Ivan Chernov, Roman Derunets, Arsenii Fomin, and Andrey Kostin. Little Brains, Big Feats: Exploring Compact Language Models. https://arxiv.org/abs/2606.30062, 2026.

[56] Arka Pal, Louai Zahran, William Gvozdjak, Akilesh Potti, and Micah Goldblum. Privacy-Preserving Mechanisms Enable Cheap Verifiable Inference of LLMs. https://arxiv.org/abs/2602.17223, 2026.